Reviewer Pack — Hypergraph Matching Complexity for Monotone DNF

Entry 5de245e2396b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 00:37:19 UTC

Hypergraph Matching Complexity for Monotone DNF

Entry ID: `5de245e2396b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-11 00:37:19 UTC

1. Conjecture

Field A (mathematical branch): Hypergraph Theory **Field B** (complexity object): Monotone DNF Formulas

Statement:

> For any monotone DNF formula with m terms over n variables, the maximum matching size μ satisfies: (i) $\mu \leq \log n + 2\sqrt{(m)}$ for $m \leq n^2$, (ii) $\mu \geq n/2$ for k-CLIQUE DNFs with $k \geq 2\sqrt{n}$.

Rationale (proposer's reasoning):

> Hypergraph matching captures structural limitations of DNFs; its growth rate could expose inherent complexity gaps between polynomial-size DNFs and k-CLIQUE's exponential requirements. The conjecture links hypergraph expansion properties to circuit size lower bounds.

Taxonomy category: `AVG_TO_WORST_CASE` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `cac96d58855a56e1`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_dnf(n, m):
        dnf = []
        for _ in range(m):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            dnf.append(clause)
        return dnf

    def is_k_clique_dnf(dnf, k):
        variables = set()
        for clause in dnf:
            variables.update(abs(var) for var in clause)
        if len(variables) < k:
            return False
        for i in range(len(variables)):
            for j in range(i + 1, len(variables)):
                if all(var not in clause for clause in dnf):
                    return True
        return False

    def greedy_matching(hypergraph):
        matching = []
        hypergraph_copy = [set(edge) for edge in hypergraph]
        while hypergraph_copy:
            max_edge_index = None
            max_size = 0
            for i, edge in enumerate(hypergraph_copy):
                if len(edge) > max_size:
                    max_size = len(edge)
                    max_edge_index = i
            matching.append(list(hypergraph_copy[max_edge_index]))
            hypergraph_copy.pop(max_edge_index)
            for i, edge in enumerate(hypergraph_copy):
                hypergraph_copy[i] -= set(matching[-1])
        return matching
```

```
    def hypergraph_from_dnf(dnf):
        hypergraph = []
        for clause in dnf:
            hypergraph.append(set(abs(var) for var in clause))
        return hypergraph

    n = random.randint(5, 40)
    m = random.randint(1, n**2)

    dnf = generate_dnf(n, m)
    hypergraph = hypergraph_from_dnf(dnf)

    matching_size = len(greedy_matching(hypergraph))

    is_k_clique = is_k_clique_dnf(dnf, math.isqrt(n) + 1)

    if is_k_clique:
        expected_min_matching_size = n // 2
    else:
        expected_max_matching_size = math.log(n, 2) + 2 * math.sqrt(m)

    conjecture_holds = (matching_size <= expected_max_matching_size and matching_size >= expected_
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Matching Size",
        "metric_value": matching_size,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    if all(result["conjecture_holds"] for result in results):
        support_fraction = 1.0
    else:
        support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(result

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(result['metric_value'] for result in results) / len(re
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5734af53.py", line 94, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5734af53.py", line 68, in run_trial
    matching_size = len(greedy_matching(hypergraph))
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5734af53.py", line 50, in greedy_matchi
    matching.append(list(hypergraph_copy[max_edge_index]))
                         ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^
TypeError: list indices must be integers or slices, not NoneType
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to TypeError, preventing result collection. Pre-registered support condition cannot be evaluated without successful trials. | next: Debug the hypergraph indexing error in test_5734af53.py and re-run experiments

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	102923
2	propose	ollama_remote	qwen3:8b	0	0	111508
3	propose	ollama_remote	qwen3:8b	0	0	90081
4	preregistration	ollama_remote	qwen3:8b	0	0	24411
5	novelty	ollama_remote	qwen3:8b	0	0	20717
6	novelty	ollama_remote	qwen3:8b	0	0	14290
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14476
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9993
9	judge	ollama_remote	qwen3:8b	0	0	18337

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 406737 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/5de245e2396b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5de245e2396b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/5de245e2396b.tar.gz` (if generated)